The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch03_dsa

AYuSh

Contents

Chapter 3: Data Structures & Algorithms (ML-flavored)

The coding round in an ML interview loop is usually a standard software-engineering screen wearing an ML costume. You will be asked to manipulate arrays and strings, count things with hash maps, keep a running top-k with a heap, traverse a graph, or write a short dynamic program — and then, in the ML-specific variant, to implement KNN, k-means, or a sampling routine from scratch with no library beyond NumPy. Interviewers use this round to answer one question: *when this person's model pipeline breaks at 2 a.m., can they reason about code, or only call* `fit()`*?*

This chapter covers the data structures and algorithms that actually appear in ML coding screens, in the order you should learn them: linear structures (arrays, strings, hash maps, stacks, queues, heaps), hierarchical and relational structures (trees, graphs, BFS/DFS), the two algorithmic workhorses (sorting and searching), dynamic programming at the level interviews test it, and finally the ML-specific implementations — KNN, k-means, and sampling algorithms — that separate ML candidates from generic software candidates. Chapter 2 covered *what Python's containers cost*; this chapter covers *what to build with them*. Throughout, every pattern is tied to where it shows up in real ML work, because that is exactly the framing a strong candidate uses out loud.

Arrays and strings

An array is a contiguous block of memory indexed in O(1); in Python interviews the list plays this role, and in ML code the NumPy ndarray is its typed, vectorized descendant. Strings are immutable character arrays — every "modification" allocates a new string, which is why building a string in a loop with `+=` is $O(n^2)$ and `''.join(parts)` is O(n). Most array and string questions are not about the structure itself but about avoiding the naive $O(n^2)$ double loop with one of three patterns.

Two pointers. Keep two indices that move through the array under some invariant. Canonical forms: pointers converging from both ends (pair-with-target-sum in a sorted array, reversing in place, container-with-most-water), and slow/fast pointers moving in the same direction (in-place deduplication of a sorted array, partitioning). The invariant is what you should narrate in an interview: "everything left of `slow` is the deduplicated prefix." Two-pointer solutions turn $O(n^2)$ pair-scans into O(n) whenever sortedness or monotonicity lets you rule out candidates wholesale.

Sliding window. A two-pointer special case for contiguous subarrays and substrings: expand the right edge, and contract the left edge only when a constraint breaks (window sum exceeds a budget, a character repeats). Each element enters and leaves the window

at most once — O(n) total. This is the pattern behind "longest substring without repeating characters," "smallest subarray with sum ≥ s," and, in ML terms, any rolling-statistic computation: a moving average over a metric stream *is* a fixed-size sliding window, and rolling means/medians over time series features are its production form.

Prefix sums. Precompute `prefix[i]` = sum of the first i elements; any range sum becomes `prefix[j] - prefix[i]` in O(1) after O(n) setup. The trade — linear preprocessing for constant-time queries — generalizes far beyond sums (prefix max, prefix counts) and is the 1-D ancestor of integral images in computer vision, which answer any rectangular patch-sum query in O(1) exactly this way.

For strings specifically, interviewers add frequency-count questions (anagrams, palindrome permutations) that are really hash-map questions, and parsing questions (tokenize, validate) that are really stack questions. Genuinely string-specific algorithms (KMP, suffix arrays) are rare in ML loops; frequency counting plus two pointers covers the large majority. The ML echo is everywhere: tokenization is string splitting with rules, vocabulary construction is frequency counting, and n-gram extraction is a sliding window over tokens.

Hash maps and sets

Chapter 2 covered how a hash table works (hash → slot → probe on collision; O(1) average, O(n) adversarial worst case). This section is about the *patterns* — a hash map is the answer to more interview questions than any other structure, because it converts "have I seen this before?" and "how many of these are there?" from O(n) scans into O(1) lookups.

Seen-before. Two-sum is the archetype: for each `x`, ask whether `target - x` was already seen. One pass, O(n) time, O(n) space — versus $O(n^2)$ for the double loop or O(n log n) for sort-then-two-pointers. The general move: when a double loop's inner scan is *searching for a value*, replace the scan with a dict.

Counting. `collections.Counter` builds a frequency table in one O(n) pass; anagram checks compare two Counters; "group anagrams" keys a dict by the sorted word (or a 26-tuple of counts). In ML this pattern is load-bearing: building a vocabulary is `Counter(tokens).most_common(vocab_size)`, class-imbalance diagnosis is `Counter(labels)`, and TF (term frequency) in TF-IDF is literally a per-document Counter.

Grouping / indexing. A dict of lists (`defaultdict(list)`) groups records by key in O(n) — the in-memory version of SQL's GROUP BY and the first half of a hash join. An inverted index (token → list of document ids) is the same pattern and is the core data structure of classical retrieval; it reappears in Chapter 24 as the sparse half of hybrid RAG search.

Two costs to volunteer before the interviewer asks: hash maps buy speed with memory (O(n) extra) and lose ordering (you cannot ask a dict for "the smallest key" without scanning — that is what heaps and sorted structures are for). And keys must be hashable:

a NumPy array cannot key a dict; convert to `tuple(arr)` or `arr.tobytes()` first — a detail that bites in memoized feature pipelines.

Stacks and queues

A **stack** is last-in-first-out: `append` and `pop` on the end of a Python list, both O(1). It is the structure of *nesting* — matched parentheses, undo histories, recursion. Every recursive algorithm implicitly uses the call stack, and converting recursion to an explicit stack is a standard interview follow-up ("now do it iteratively") as well as the practical fix for `RecursionError` on deep structures — a depth-10,000 decision tree will blow Python's default ~1,000-frame recursion limit during traversal, and the cure is an explicit stack. The framework echo is exact: autodiff engines record operations on a tape and pop them in reverse during backpropagation — the backward pass is a stack traversal of the forward computation.

The classic hard-stack question is the **monotonic stack**: maintain a stack whose elements are kept sorted by popping everything that violates the order before each push ("next greater element," "largest rectangle in histogram"). Each element is pushed and popped at most once — O(n) for what looks like an O(n²) problem.

A **queue** is first-in-first-out: use `collections.deque` (`append` / `popleft`, both O(1)); `list.pop(0)` is O(n) and naming that cost is a free point in interviews. Queues are the structure of *fair processing in arrival order*: BFS frontiers (next section), producer-consumer buffers. That producer-consumer shape is exactly a training input pipeline — DataLoader workers put prepared batches on a queue, the training loop gets them; the queue decouples the two speeds. A bounded deque (`deque(maxlen=k)`) is the natural container for "last k checkpoints" or a rolling window of recent losses.

Heaps and priority queues

A heap answers a question neither a dict nor a sorted list answers well: *repeatedly give me the smallest (or largest) item as items keep arriving*. A binary min-heap is a complete binary tree, stored flat in an array (children of index i at 2i+1 and 2i+2 — no pointers), with one invariant: every parent ≤ its children. The root is the minimum, readable in O(1); `push` and `pop` restore the invariant by bubbling an element up or down one level at a time, O(log n) each; building a heap from n items in one shot (`heapify`) is O(n), cheaper than n pushes.

Python's `heapq` is a min-heap over lists. Two idioms to know cold: for a max-heap, push negated values (or negate a numeric key); for compound items, push tuples `(key, payload)` — heapq compares tuples lexicographically, so ties on the key fall through to comparing payloads, which crashes if payloads aren't comparable; insert a tiebreaker counter `(key, i, payload)`.

Top-k, the signature ML pattern. To keep the k largest of a stream: maintain a *min*-heap of size k; each new item is compared against the heap's root (the smallest of the current top-k) and replaces it if larger — `heapq.heappushpop` does this in one O(log k) step. Total O(n log k) time, O(k) memory, versus O(n log n) and O(n) for full sorting. The inversion — a min-heap to track *largest* items — is the part candidates fumble; say out loud that the root is "the weakest member of the club, first to be evicted." This pattern is everywhere in ML: k nearest neighbors from a distance stream, beam search (keep the k best partial sequences per step — Chapter 22), top-k tokens before sampling, best-k checkpoints by validation loss. `heapq.nlargest(k, xs)` packages it.

Merge k sorted streams. A heap of size k holding the current head of each stream pops the global minimum in O(log k) — the interview form is "merge k sorted lists," the systems form is external-merge sort over shards too big for memory (Chapter 28's territory). Dijkstra's shortest-path algorithm is the same idea: a priority queue always expands the closest unexplored node.

Trees

A tree is a connected graph with no cycles: n nodes, n−1 edges, one root, every non-root node with exactly one parent. Interview trees are usually **binary trees** (≤ 2 children), and the vocabulary is fair game: *depth* of a node (edges from root), *height* of the tree, *leaf*, *balanced* (heights of any node's subtrees differ by at most 1). A **binary search tree (BST)** adds the ordering invariant — everything in the left subtree < node < everything in the right subtree — which makes search, insert, and delete O(h), where h is the height: O(log n) if balanced, degrading to O(n) if the tree becomes a linked list (insert sorted data into a naive BST and it does exactly that; production structures like red-black trees rebalance to keep h logarithmic, and knowing *that they exist and why* is the expected depth).

Traversals are the heart of tree questions, and nearly all are four lines of recursion. Depth-first comes in three orders, named for where the node is visited relative to its children: **preorder** (node, left, right — serializing/copying a tree), **inorder** (left, node, right — which on a BST yields sorted order, the single most-tested tree fact), and **postorder** (left, right, node — children before parents: deleting a tree, computing subtree aggregates). **Level-order** is breadth-first with a queue. Most "hard" tree questions are one traversal plus bookkeeping: validate a BST (inorder must be increasing, or pass down min/max bounds), max depth (postorder), lowest common ancestor.

Trees are load-bearing in ML. A decision tree *is* a binary tree — prediction is root-to-leaf descent by feature thresholds, O(depth); training (Chapter 6) is recursive splitting, a build-the-tree recursion; and feature importances are computed by a postorder-style aggregation over split gains. Gradient-boosted forests are collections of hundreds of such trees. KD-trees and ball trees partition feature space to accelerate nearest-neighbor

search from O(n) toward O(log n) per query in low dimensions — sklearn's KNeighborsClassifier chooses among brute force, KD-tree, and ball tree — though in high dimensions partitioning stops pruning anything (the curse of dimensionality, Chapter 4) and brute force or approximate methods (Chapter 24's HNSW) win. Parse trees and abstract syntax trees are how NLP represented structure before transformers flattened everything into attention.

Graphs, BFS and DFS

A graph is nodes plus edges — directed or undirected, weighted or not, possibly cyclic. Interviews expect two representations and their trade-off: the **adjacency list** (dict mapping node → list of neighbors; O(V+E) space; the default) and the **adjacency matrix** (V×V array; O(V^2) space; O(1) edge lookup; only sensible for dense or small graphs). Real graphs are usually sparse, so adjacency lists win. Two special cases matter: a tree is a connected acyclic graph, and a **DAG** (directed acyclic graph) is the shape of every dependency system — including every neural network's computation graph and every feature pipeline.

BFS explores level by level with a queue: pop a node, push unvisited neighbors, repeat. Because it expands outward one edge-layer at a time, BFS finds *shortest paths in unweighted graphs* — that is its superpower and the reason to choose it. **DFS** dives deep with recursion or an explicit stack, and is the natural fit for exhaustive exploration: connected components, cycle detection, backtracking, topological sort. Both are O(V+E) time; both require a `visited` set, and forgetting it is the classic infinite-loop bug on any graph with a cycle (on trees you can skip it only because trees have none). Choose by question: "shortest/fewest steps" → BFS; "all paths / does a path exist / components / ordering" → DFS. Memory differs too — BFS holds a frontier (up to O(V) wide), DFS holds a path (up to O(V) deep, hence recursion limits on long chains).

Topological sort linearizes a DAG so every edge points forward: either DFS-with-postorder-reversal, or Kahn's algorithm (repeatedly remove zero-in-degree nodes using a queue; if nodes remain un-removed, the graph has a cycle — which is how you *detect* cycles in dependency systems). This is the algorithm ML infrastructure runs on: a feature pipeline's "compute features before the model that consumes them" is a topological order; Airflow schedules DAG tasks this way (Chapter 28); and autodiff frameworks execute the forward pass in topological order of the computation graph, then backpropagate in exactly the reverse order (Chapter 12). "Backprop visits nodes in reverse topological order" is a sentence that upgrades a coding answer into an ML answer.

Weighted shortest paths (Dijkstra — BFS with a priority queue instead of a queue) and connectivity under merging (union-find) are worth recognizing by name, but ML loops rarely go deeper than BFS/DFS/topo-sort plus their applications: label propagation over a

similarity graph is BFS flood-fill, and graph neural networks (Chapter 25's neighborhood) aggregate neighbors — one BFS layer per GNN layer.

Sorting and searching

You will almost never implement a sort in production — `sorted()` exists — but interviews test sorting because it is the cleanest window into algorithmic reasoning, and one implementation question ("implement quicksort/mergesort") is common as a warm-up.

The comparison-sort landscape in one pass: **mergesort** splits in half, sorts each, and merges — O(n log n) *guaranteed*, stable, but O(n) extra memory; it is the shape of external sorting when data exceeds RAM. **Quicksort** picks a pivot, partitions smaller/larger, and recurses — O(n log n) average with excellent constants and in-place partitioning, but $O(n^2)$ worst case on adversarial pivots (mitigated by random pivot choice). **Heapsort** is O(n log n) guaranteed and in-place but slower in practice and unstable. Python's built-in **Timsort** is a mergesort-insertion hybrid exploiting existing runs: O(n log n) worst case, O(n) on nearly-sorted data, stable. **Stability** — equal keys keep their original order — is the practically important property: it is what makes multi-key sorting composable (sort by secondary key, then stable-sort by primary) and preserves temporal order when you sort event logs by user, which matters for leakage-free time splits (Chapter 4). Counting-based sorts (counting/radix/bucket) beat O(n log n) by not comparing at all, and O(n log n) is a proven lower bound for comparison sorts — a classic follow-up.

Two sorting-adjacent tools earn their keep in ML code. **argsort** — sort the *indices*, not the values — is how you rank one array by another: `np.argsort(scores)[::-1]` orders candidates by score, `np.argsort` on distances gives nearest neighbors, and feature importances are displayed via argsort. **Quickselect** finds the k-th smallest element in O(n) average by running quicksort's partition but recursing into only one side — `np.partition`/`np.argpartition` expose it, and it is the right way to get top-k when k is large or to compute a median (robust statistics, Chapter 9's outlier handling) without a full sort.

Binary search is the highest-value algorithm per line of code in this chapter: on any sorted (more generally, *monotonic*) structure, halve the search space per step, O(log n). The implementation is famously bug-prone — the interview-safe invariant form keeps `[lo, hi)` as "the range that could still contain the answer," computes `mid = (lo + hi) // 2`, and moves one boundary per step; off-by-one errors and infinite loops come from mixing inclusive and exclusive conventions mid-function. `bisect.bisect_left/right` are the library forms and also the tool for "insert into sorted list" and "count items ≤ x" (which is how you binary-search a CDF — see weighted sampling below). The pattern's real power is **binary search on the answer**: if a yes/no feasibility check is monotonic in some parameter, binary-search the parameter itself — "smallest capacity that ships packages in D days" in interview form; threshold tuning against a monotonic constraint ("largest

classification threshold with recall ≥ 0.9" — precision-recall tradeoffs, Chapter 10) and learning-rate range probes in ML form. Say the magic word *monotonic* — it is the property that licenses the halving.

Dynamic programming basics

Dynamic programming applies when a problem has two properties: **optimal substructure** (the best solution is composed of best solutions to subproblems) and **overlapping subproblems** (naive recursion solves the same subproblem exponentially many times). DP is recursion plus a cache — nothing more mystical than that. Fibonacci is the toy case: naive recursion is $O(2^n)$ because fib(k) is recomputed everywhere; caching results makes it O(n).

Two mechanical styles. **Top-down (memoization):** write the natural recursion, add `@functools.lru_cache` (Chapter 2's decorator, now load-bearing). **Bottom-up (tabulation):** fill a table from base cases upward with a loop, which avoids recursion limits and often allows compressing the table to its last row or two — O(n) space becoming O(1) for Fibonacci-shaped recurrences. Interviewers commonly accept either and then ask for the space optimization.

The recipe that solves most interview DP: (1) define the state in words — "dp[i] = best value achievable using the first i items"; (2) write the recurrence — how dp[i] follows from smaller states; (3) identify base cases; (4) fix the fill order so dependencies are ready; (5) read off the answer. The classic families, each worth one worked pass: 1-D sequence DP (climbing stairs, house robber — dp over a prefix), **longest common subsequence / edit distance** (2-D table over two sequence prefixes), and **0/1 knapsack** (choose or skip each item under a budget — the shape of feature selection under a cost constraint, though in practice we use greedier methods).

Edit distance is the one to know deeply, because it is genuinely used: `dist(i, j)` = cost of turning the first i characters of one string into the first j of another; if the current characters match, inherit `dist(i-1, j-1)`, else 1 + min(insert, delete, substitute). It powers spelling correction, fuzzy string matching in data cleaning and entity resolution, and — in bioinformatics dress — sequence alignment. Its deeper ML relatives share the exact same table-filling structure: dynamic time warping aligns two time series (Chapter 31), the Viterbi algorithm finds the best hidden-state path in an HMM (Chapter 19), and CTC loss for speech recognition marginalizes over alignments — all are "fill a 2-D table by a local recurrence." Beam search (Chapter 22) is what you do when the exact DP is intractable: keep only the k best states per column. And the Bellman equation of reinforcement learning (Chapter 32) *is* a DP recurrence over states — the term "dynamic programming" comes from Bellman himself.

ML-specific coding: KNN, k-means, and sampling from scratch

The ML round's favorite question type: "implement X from scratch, NumPy only." It tests whether you understand the algorithm well enough to state it as array operations, and whether Chapter 2's vectorization reflexes are real. Three families dominate; the full implementations, executed with output, are in the listings below.

KNN (Listing 9) is pure computation, no training: predict by majority vote (or average, for regression) of the k nearest training points. The from-scratch skeleton is: pairwise distances (vectorized via broadcasting or the expansion of the squared-distance formula), `argpartition` for the k smallest per query — an O(n) selection rather than an O(n log n) sort, exactly quickselect earning its keep — then a vote. Complexity is the punchline interviewers want: O(n·d) *per query* at prediction time and O(1) training, the inverse of parametric models; the space-partitioning escape hatches (KD-trees, and their high-dimensional failure) connect back to the trees section.

K-means (Listing 10) is the two-step dance: assign every point to its nearest centroid, recompute each centroid as the mean of its assigned points, repeat until assignments stop changing. Each step is one broadcasting expression; the pedagogical payload is that both steps *provably* never increase the within-cluster sum of squares, so the loop converges — to a local optimum that depends on initialization (hence k-means++ and multiple restarts, Chapter 8). Edge case to handle out loud: a centroid that loses all its points (reseed it randomly).

Sampling algorithms are the sleeper topic — they look niche but appear constantly, because training *is* sampling. Four are interview staples. *Fisher–Yates shuffle* (Listing 11): swap position i with a uniformly random position in [i, n) — O(n), exactly uniform over permutations; it is what every `shuffle=True` runs, and the naive "sort by random key" alternative is O(n log n) while "swap with a random position in [0, n)" is subtly *non*-uniform, a known trap. *Reservoir sampling* (Listing 11): keep a uniform sample of k items from a stream of unknown length — keep the first k, then accept item i with probability k/i, evicting a random resident; the proof by induction that every item ends up with probability k/n is a classic whiteboard follow-up; production form: sampling from a data stream too large to hold. *Weighted sampling* (Listing 12): build the cumulative distribution and binary-search a uniform random number into it — O(log n) per draw after O(n) setup; this is temperature sampling from a softmax (Chapter 22), boosting's example reweighting (Chapter 7), and prioritized replay (Chapter 32). *Stratified splitting* (Listing 12): group indices by label (a hash-map grouping), sample within each group — the correct train/test split under class imbalance (Chapter 4).

Code implementations

All listings below were executed as shown; the output blocks are real. Run them, break them, and re-derive them — in a live screen you will be writing these from memory.

Listing 1 — Two pointers and sliding window

Four O(n) patterns that replace O(n^2) double loops. Narrate the invariant: everything left of `slow` is finished work; the window `[left, right]` always satisfies (or has just broken) the constraint.

```
"""Listing 1 -- Two pointers and sliding window."""

def dedup_sorted(a):
    """In-place dedup of a sorted list. Invariant: a[:slow] is the deduplicated
prefix."""
    if not a:
        return 0
    slow = 1                            # next write position
    for fast in range(1, len(a)):       # fast scans every element
        if a[fast] != a[slow - 1]:      # new value -> keep it
            a[slow] = a[fast]
            slow += 1
    return slow                         # length of deduplicated prefix

def pair_with_sum(a, target):
    """Sorted array: find a pair summing to target. Converging pointers, O(n)."""
    lo, hi = 0, len(a) - 1
    while lo < hi:
        s = a[lo] + a[hi]
        if s == target:
            return a[lo], a[hi]
        if s < target:                  # sum too small -> only moving lo up can help
            lo += 1
        else:                           # sum too big -> only moving hi down can help
            hi -= 1
    return None

def smallest_window_geq(a, s):
    """Length of the smallest contiguous window with sum >= s. O(n)."""
    best = float('inf')
    left = wsum = 0
    for right, x in enumerate(a):       # expand right edge
        wsum += x
        while wsum >= s:                # contract while constraint satisfied
            best = min(best, right - left + 1)
            wsum -= a[left]
            left += 1
    return best if best != float('inf') else 0

def rolling_mean(xs, k):
    """Moving average via a running window sum -- O(n), not O(n*k)."""
    out, wsum = [], 0.0
    for i, x in enumerate(xs):
        wsum += x
        if i >= k:
            wsum -= xs[i - k]           # element leaves the window exactly once
        if i >= k - 1:
            out.append(wsum / k)
    return out

a = [1, 1, 2, 3, 3, 3, 5]
```

```
n = dedup_sorted(a)
print("dedup:", a[:n])
print("pair summing to 8 in [1,2,4,6,7]:", pair_with_sum([1, 2, 4, 6, 7], 8))
print("smallest window >= 7 in [2,3,1,2,4,3]:", smallest_window_geq([2, 3, 1, 2, 4,
3], 7))
print("rolling mean k=3 of [1..5]:", rolling_mean([1, 2, 3, 4, 5], 3))
```

Output:

```
dedup: [1, 2, 3, 5]
pair summing to 8 in [1,2,4,6,7]: (1, 7)
smallest window >= 7 in [2,3,1,2,4,3]: 2
rolling mean k=3 of [1..5]: [2.0, 3.0, 4.0]
```

Listing 2 — Hash-map patterns: seen-before, counting, grouping

The three moves that answer most hash-map questions, ending with the ML form: vocabulary construction is the counting pattern verbatim.

```
"""Listing 2 -- Hash-map patterns: seen-before, counting, grouping."""
from collections import Counter, defaultdict

def two_sum(nums, target):
    """Indices of two numbers summing to target. One pass, O(n) time/space."""
    seen = {}                           # value -> index
    for i, x in enumerate(nums):
        if target - x in seen:          # the O(1) lookup that replaces the inner loop
            return seen[target - x], i
        seen[x] = i
    return None

def group_anagrams(words):
    """Key each word by its sorted letters; anagrams collide on the same key."""
    groups = defaultdict(list)
    for w in words:
        groups[''.join(sorted(w))].append(w)
    return list(groups.values())

def build_vocab(tokens, size):
    """Counting pattern: top-`size` tokens by frequency -> token->id map."""
    counts = Counter(tokens)                        # one O(n) pass
    most = counts.most_common(size)                 # heap-based top-k inside
    return {tok: i for i, (tok, _) in enumerate(most)}

print("two_sum([2,7,11,15], 9):", two_sum([2, 7, 11, 15], 9))
print("anagram groups:", group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
toks = "the cat sat on the mat the cat".split()
print("vocab (size 3):", build_vocab(toks, 3))
```

Output:

```
two_sum([2,7,11,15], 9): (0, 1)
anagram groups: [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
vocab (size 3): {'the': 0, 'cat': 1, 'sat': 2}
```

Listing 3 — Heaps: streaming top-k and merge-k

The signature inversion — a *min*-heap tracks the *largest* k items, its root the weakest member and first to be evicted. Tuples `(value, list_id, index)` give heapq a total order with no custom comparator.

```
"""Listing 3 -- Heaps: streaming top-k and merging k sorted streams."""
import heapq

def top_k_stream(stream, k):
    """Keep the k largest items of a stream. Min-heap of size k, O(n log k)."""
    heap = []
    for x in stream:
        if len(heap) < k:
            heapq.heappush(heap, x)
        elif x > heap[0]:                   # beats the weakest current member?
            heapq.heappushpop(heap, x)      # evict root, insert x -- one O(log k) op
    return sorted(heap, reverse=True)

def merge_k_sorted(lists):
    """Merge k sorted lists. Heap holds one head per list; pop = global min."""
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]  # (value, list_id,
index)
    heapq.heapify(heap)                     # O(k)
    out = []
    while heap:
        val, i, j = heapq.heappop(heap)     # O(log k)
        out.append(val)
        if j + 1 < len(lists[i]):
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out

scores = [0.11, 0.93, 0.35, 0.88, 0.62, 0.99, 0.41, 0.77]
print("top-3 scores:", top_k_stream(scores, 3))
print("merge:", merge_k_sorted([[1, 4, 9], [2, 3, 11], [5, 6, 7]]))
```

Output:

```
top-3 scores: [0.99, 0.93, 0.88]
merge: [1, 2, 3, 4, 5, 6, 7, 9, 11]
```

Listing 4 — Tree traversals and BST validation

Each traversal is four lines; the payload is knowing which order fits which job. The corruption test shows why 'check each node against its children' is the classic wrong answer — validity is a *subtree-wide* bounds condition.

```
"""Listing 4 -- Tree traversals and BST validation."""
from collections import deque

class Node:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def inorder(node, out):
    """left, node, right -- sorted order on a BST."""
    if node:
        inorder(node.left, out)
```

```
        out.append(node.val)
        inorder(node.right, out)

def max_depth(node):
    """Postorder aggregation: children first, then combine."""
    if node is None:
        return 0
    return 1 + max(max_depth(node.left), max_depth(node.right))

def level_order(root):
    """BFS with a queue; one output list per level."""
    if not root:
        return []
    levels, q = [], deque([root])
    while q:
        levels.append([n.val for n in q])
        q = deque(c for n in q for c in (n.left, n.right) if c)
    return levels

def is_bst(node, lo=float('-inf'), hi=float('inf')):
    """Pass down the (lo, hi) bounds each node must satisfy."""
    if node is None:
        return True
    if not (lo < node.val < hi):
        return False
    return is_bst(node.left, lo, node.val) and is_bst(node.right, node.val, hi)

#        8
#      /   \
#     3     10
#    / \      \
#   1   6      14
root = Node(8, Node(3, Node(1), Node(6)), Node(10, None, Node(14)))
out = []
inorder(root, out)
print("inorder:", out)            # sorted <=> valid BST
print("max depth:", max_depth(root))
print("levels:", level_order(root))
print("is_bst:", is_bst(root))
root.left.right.val = 9           # 9 > 8 but sits in the left subtree
print("is_bst after corruption:", is_bst(root))
```

Output:

```
inorder: [1, 3, 6, 8, 10, 14]
max depth: 3
levels: [[8], [3, 10], [1, 6, 14]]
is_bst: True
is_bst after corruption: False
```

Listing 5 — Graphs: BFS shortest path, DFS components, topological sort

The visited set does double duty as the BFS distance map. DFS is iterative with an explicit stack — no recursion limit. Kahn's algorithm both orders the feature-pipeline DAG and detects the cycle when a back-edge is added.

```
"""Listing 5 -- BFS shortest path, DFS components, Kahn's topological sort."""
from collections import deque, defaultdict

def bfs_shortest(adj, src, dst):
```

```
    """Fewest-edges path length in an unweighted graph."""
    q, dist = deque([src]), {src: 0}
    while q:
        u = q.popleft()
        if u == dst:
            return dist[u]
        for v in adj[u]:
            if v not in dist:              # `dist` doubles as the visited set
                dist[v] = dist[u] + 1
                q.append(v)
    return -1

def components(adj, nodes):
    """Connected components via iterative DFS (explicit stack, no recursion limit)."""
    seen, comps = set(), []
    for s in nodes:
        if s in seen:
            continue
        stack, comp = [s], []
        seen.add(s)
        while stack:
            u = stack.pop()
            comp.append(u)
            for v in adj[u]:
                if v not in seen:
                    seen.add(v)
                    stack.append(v)
        comps.append(sorted(comp))
    return comps

def topo_sort(edges):
    """Kahn's algorithm. Returns order, or raises if a cycle exists."""
    adj, indeg = defaultdict(list), defaultdict(int)
    nodes = set()
    for u, v in edges:
        adj[u].append(v)
        indeg[v] += 1
        nodes |= {u, v}
    q = deque(sorted(n for n in nodes if indeg[n] == 0))
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    if len(order) != len(nodes):
        raise ValueError("cycle detected -- not a DAG")
    return order

adj = {1: [2, 3], 2: [1, 4], 3: [1], 4: [2], 5: [6], 6: [5]}
print("shortest 3->4:", bfs_shortest(adj, 3, 4))
print("components:", components(adj, sorted(adj)))

# A feature pipeline as a DAG: raw -> cleaned -> {features, labels} -> model
pipeline = [("raw", "cleaned"), ("cleaned", "features"),
            ("cleaned", "labels"), ("features", "model"), ("labels", "model")]
print("pipeline order:", topo_sort(pipeline))
try:
    topo_sort(pipeline + [("model", "raw")])
except ValueError as e:
    print("with back-edge:", e)
```

Output:

```
shortest 3->4: 3
components: [[1, 2, 3, 4], [5, 6]]
pipeline order: ['raw', 'cleaned', 'features', 'labels', 'model']
with back-edge: cycle detected -- not a DAG
```

Listing 6 — Binary search: the invariant form and search on the answer

The half-open `[lo, hi)` convention removes the off-by-one bugs; `smallest_capacity` shows the pattern's real power — binary-searching a *parameter* whose feasibility check is monotonic.

```python
"""Listing 6 -- Binary search: the invariant form, and search on the answer."""
import bisect

def binary_search(a, target):
    """Half-open invariant: the answer, if present, is always in [lo, hi)."""
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1          # mid ruled out -> exclude it
        else:
            hi = mid              # a[mid] too big; hi stays exclusive
    return -1

def smallest_capacity(weights, days):
    """Search on the answer: min capacity to ship `weights` in `days` days.
    feasible(c) is monotonic in c -- that is what licenses the halving."""
    def feasible(cap):
        d, load = 1, 0
        for w in weights:
            if load + w > cap:
                d, load = d + 1, 0
            load += w
        return d <= days
    lo, hi = max(weights), sum(weights)   # answer bracketed in [lo, hi]
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid):
            hi = mid              # mid works -> try smaller
        else:
            lo = mid + 1          # mid fails -> need bigger
    return lo

a = [2, 5, 8, 12, 16, 23, 38]
print("index of 16:", binary_search(a, 16))
print("index of 17:", binary_search(a, 17))
print("insertion point for 17:", bisect.bisect_left(a, 17))
print("min capacity, [1..10] in 5 days:", smallest_capacity(list(range(1, 11)), 5))
```

Output:

```
index of 16: 4
index of 17: -1
insertion point for 17: 5
min capacity, [1..10] in 5 days: 15
```

Listing 7 — Mergesort and quickselect

Mergesort as the canonical guaranteed-O(n log n) stable sort (the `<=` in the merge is where stability lives); quickselect finds the median in O(n) average by recursing into only one partition.

```
"""Listing 7 -- Mergesort and quickselect."""
import random

def mergesort(a):
    """O(n log n) guaranteed, stable, O(n) extra memory."""
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left, right = mergesort(a[:mid]), mergesort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:       # `<=` is what makes the sort stable
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

def quickselect(a, k):
    """k-th smallest (0-indexed), O(n) average: partition, recurse one side only."""
    if len(a) == 1:
        return a[0]
    pivot = random.choice(a)          # random pivot defends against O(n^2)
    lt = [x for x in a if x < pivot]
    eq = [x for x in a if x == pivot]
    gt = [x for x in a if x > pivot]
    if k < len(lt):
        return quickselect(lt, k)
    if k < len(lt) + len(eq):
        return pivot
    return quickselect(gt, k - len(lt) - len(eq))

random.seed(0)
xs = [7, 2, 9, 4, 4, 1, 8, 3]
print("mergesort:", mergesort(xs))
print("median via quickselect:", quickselect(xs, len(xs) // 2))
```

Output:

```
mergesort: [1, 2, 3, 4, 4, 7, 8, 9]
median via quickselect: 4
```

Listing 8 — Dynamic programming: memoized Fibonacci and edit distance

Top-down DP is one decorator; bottom-up edit distance is the 2-D table every alignment algorithm (DTW, Viterbi, CTC) generalizes. `fib(80)` would take ~2^80 calls naively; memoized it takes 81.

```
"""Listing 8 -- Dynamic programming: memoized Fibonacci and edit distance."""
from functools import lru_cache
```

```
@lru_cache(maxsize=None)              # recursion + cache = top-down DP
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

def edit_distance(s, t):
    """dp[i][j] = min ops to turn s[:i] into t[:j]. Bottom-up, O(len(s)*len(t))."""
    m, n = len(s), len(t)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i                  # delete all of s[:i]
    for j in range(n + 1):
        dp[0][j] = j                  # insert all of t[:j]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s[i - 1] == t[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]          # match: free
            else:
                dp[i][j] = 1 + min(dp[i - 1][j],     # delete from s
                                   dp[i][j - 1],     # insert into s
                                   dp[i - 1][j - 1]) # substitute
    return dp[m][n]

print("fib(80):", fib(80))            # naive recursion: ~2^80 calls; memoized: 81
print("edit('kitten','sitting'):", edit_distance("kitten", "sitting"))
print("edit('gradient','gradinet'):", edit_distance("gradient", "gradinet"))
```

Output:

```
fib(80): 23416728348467685
edit('kitten','sitting'): 3
edit('gradient','gradinet'): 2
```

Listing 9 — KNN from scratch

No training loop, no Python loops over points: pairwise distances via the squared-norm expansion, k-smallest via `argpartition` (O(n) selection), then a vote. 96% held-out accuracy on two Gaussian blobs confirms it works.

```
"""Listing 9 -- KNN from scratch, NumPy only."""
import numpy as np

def knn_predict(X_train, y_train, X_query, k):
    """Vectorized k-nearest-neighbors classification.
    Distances via the expansion ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a.b -- no loops."""
    # (q, n) matrix of squared distances between every query and training point
    d2 = (np.sum(X_query**2, axis=1)[:, None]      # (q, 1)
          + np.sum(X_train**2, axis=1)[None, :]    # (1, n)  -> broadcasts to (q, n)
          - 2 * X_query @ X_train.T)               # (q, n)
    # k smallest per row: argpartition is O(n) selection, not an O(n log n) sort
    nn = np.argpartition(d2, k, axis=1)[:, :k]     # (q, k) neighbor indices
    votes = y_train[nn]                            # (q, k) neighbor labels
    # majority vote per query row
    return np.array([np.bincount(row).argmax() for row in votes])

rng = np.random.default_rng(42)
# two Gaussian blobs: class 0 near (0,0), class 1 near (3,3)
X0 = rng.normal(0, 1, (50, 2)); X1 = rng.normal(3, 1, (50, 2))
X = np.vstack([X0, X1]); y = np.array([0]*50 + [1]*50)
Xq = np.array([[0.0, 0.0], [3.0, 3.0], [1.5, 1.5]])
print("predictions:", knn_predict(X, y, Xq, k=5))
```

```
# sanity: accuracy on held-out points from the same blobs
Xt0 = rng.normal(0, 1, (100, 2)); Xt1 = rng.normal(3, 1, (100, 2))
Xt = np.vstack([Xt0, Xt1]); yt = np.array([0]*100 + [1]*100)
acc = (knn_predict(X, y, Xt, k=5) == yt).mean()
print(f"held-out accuracy: {acc:.3f}")
```

Output:

```
predictions: [0 1 1]
held-out accuracy: 0.960
```

Listing 10 — k-means from scratch

Assign and update are each one broadcasting expression. Both steps provably never increase inertia, so the loop converges — to a local optimum. Note the empty-cluster reseed, the edge case interviewers probe.

```
"""Listing 10 -- k-means from scratch, NumPy only."""
import numpy as np

def kmeans(X, k, n_iter=100, seed=0):
    rng = np.random.default_rng(seed)
    # initialize centroids as k random distinct data points
    centroids = X[rng.choice(len(X), size=k, replace=False)]
    for _ in range(n_iter):
        # ASSIGN: nearest centroid for every point. (n,1,d)-(k,d) -> (n,k,d)
        d2 = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)  # (n, k)
        labels = d2.argmin(axis=1)                                       # (n,)
        # UPDATE: each centroid = mean of its assigned points
        new_centroids = centroids.copy()
        for j in range(k):
            members = X[labels == j]
            if len(members):
                new_centroids[j] = members.mean(axis=0)
            else:                       # empty cluster: reseed at a random point
                new_centroids[j] = X[rng.integers(len(X))]
        if np.allclose(new_centroids, centroids):   # assignments stable -> converged
            break
        centroids = new_centroids
    inertia = ((X - centroids[labels]) ** 2).sum() # within-cluster sum of squares
    return centroids, labels, inertia

rng = np.random.default_rng(7)
blobs = np.vstack([rng.normal(m, 0.5, (60, 2)) for m in [(0, 0), (4, 0), (2, 4)]])
centroids, labels, inertia = kmeans(blobs, k=3, seed=1)
print("centroids:\n", np.round(centroids[np.argsort(centroids[:, 0])], 2))
print("cluster sizes:", np.bincount(labels), " inertia:", round(inertia, 1))
```

Output:

```
centroids:
 [[-0.09 -0.05]
 [ 1.97  4.02]
 [ 3.95 -0.1 ]]
cluster sizes: [60 60 60]  inertia: 74.5
```

Listing 11 — Fisher-Yates shuffle and reservoir sampling

The `[i, n)` swap range is exactly what makes Fisher-Yates uniform — `[0, n)` is the famous biased variant. The empirical check confirms reservoir sampling's guarantee: every item lands in a k=2-of-5 sample at ~2/5 = 40%.

```
"""Listing 11 -- Fisher-Yates shuffle and reservoir sampling."""
import random
from collections import Counter

def fisher_yates(a, rng=random):
    """Uniform in-place shuffle: swap i with a random j in [i, n)."""
    for i in range(len(a) - 1):
        j = rng.randrange(i, len(a))   # NOT range(0, n) -- that version is biased
        a[i], a[j] = a[j], a[i]
    return a

def reservoir_sample(stream, k, rng=random):
    """Uniform k-sample from a stream of unknown length, O(k) memory."""
    reservoir = []
    for i, x in enumerate(stream, start=1):
        if i <= k:
            reservoir.append(x)                # keep the first k
        elif rng.random() < k / i:             # accept item i with prob k/i
            reservoir[rng.randrange(k)] = x    # evict a uniformly random resident
    return reservoir

random.seed(0)
print("shuffle:", fisher_yates(list(range(10))))
print("reservoir (k=3) from range(1000):", reservoir_sample(iter(range(1000)), 3))

# empirical uniformity check: each of 5 items should land in a k=2 sample ~40%
counts = Counter()
for _ in range(20000):
    counts.update(reservoir_sample(iter(range(5)), 2))
print("inclusion frequencies:", {i: round(c / 20000, 3) for i, c in
sorted(counts.items())})
```

Output:

```
shuffle: [6, 7, 2, 5, 8, 4, 9, 3, 0, 1]
reservoir (k=3) from range(1000): [249, 584, 704]
inclusion frequencies: {0: 0.399, 1: 0.4, 2: 0.397, 3: 0.397, 4: 0.407}
```

Listing 12 — Weighted sampling and stratified splitting

Weighted draws = cumulative sums + `bisect` — the same mechanism as temperature sampling from a softmax. Stratified splitting is the hash-map grouping pattern applied to leakage-free evaluation: the 80/20 class ratio survives the split exactly.

```
"""Listing 12 -- Weighted sampling via CDF + binary search; stratified split."""
import bisect, random
from collections import Counter, defaultdict
from itertools import accumulate

def weighted_sampler(weights, rng=random):
    """O(n) setup, O(log n) per draw."""
    cdf = list(accumulate(weights))            # cumulative sums
```

```
    total = cdf[-1]
    def draw():
        u = rng.random() * total               # uniform in [0, total)
        return bisect.bisect_right(cdf, u)     # first index with cdf > u
    return draw

def stratified_split(labels, test_frac, rng=random):
    """Per-class index split -- preserves class proportions."""
    by_class = defaultdict(list)               # hash-map grouping pattern
    for i, y in enumerate(labels):
        by_class[y].append(i)
    train, test = [], []
    for y, idxs in by_class.items():
        rng.shuffle(idxs)
        cut = int(len(idxs) * test_frac)
        test += idxs[:cut]
        train += idxs[cut:]
    return sorted(train), sorted(test)

random.seed(0)
draw = weighted_sampler([0.1, 0.2, 0.7])       # e.g. softmax probabilities
freq = Counter(draw() for _ in range(10000))
print("empirical draw frequencies:", {i: round(c / 10000, 3) for i, c in
sorted(freq.items())})

labels = [0]*80 + [1]*20                       # 80/20 imbalance
train, test = stratified_split(labels, test_frac=0.25)
print("test class balance:", Counter(labels[i] for i in test))
print("train class balance:", Counter(labels[i] for i in train))
```

Output:

```
empirical draw frequencies: {0: 0.103, 1: 0.192, 2: 0.705}
test class balance: Counter({0: 20, 1: 5})
train class balance: Counter({0: 60, 1: 15})
```

Pitfalls, comparisons and practical tips

Structure selection at a glance. The interview meta-skill is hearing a requirement and naming the structure in one step:

You need...	Use	Cost
"Have I seen this?" / count things	set / dict (Counter)	O(1) avg per op
Running min/max of a stream, top-k	heap	O(log k) per op
Process in arrival order / level order	deque (queue)	O(1) per op
Most-recent-first, nesting, undo	list (stack)	O(1) per op
Membership in *sorted* data, thresholds	sorted array + bisect	O(log n) lookup
Fewest steps between states	BFS	O(V+E)
Dependency ordering / cycle check	topological sort	O(V+E)
k-th smallest / median without sorting	quickselect / np.partition	O(n) avg

Complexity traps that cost offers. `list.pop(0)` and `list.insert(0, x)` are O(n) — use deque. `x in my_list` is O(n) inside a loop — that is an accidental $O(n^2)$; convert to a set once. String `+=` in a loop is $O(n^2)$ — collect parts, `''.join`. Slicing copies: `a[1:]` in a recursion turns O(n) algorithms into $O(n^2)$ and is the classic hidden cost in tidy-looking recursive solutions — pass indices instead. `sorted()` inside a loop over n iterations is $O(n^2 \log n)$ — sort once outside, or maintain a heap.

Recursion traps. Python's default recursion limit (~1,000) converts deep recursions into crashes: DFS on a long path graph, traversal of a degenerate tree, divide-and-conquer on adversarial input. The fixes, in preference order: rewrite with an explicit stack (Listing 5's DFS), convert the DP to bottom-up tabulation, or — last resort — raise the limit. Also remember Chapter 2's mutable-default trap: `def dfs(node, visited=set())` shares one set across *calls*, a bug that surfaces exactly in graph code.

Off-by-one discipline for binary search. Pick the half-open `[lo, hi)` convention and never mix it with inclusive bounds mid-function. Symptoms of mixing: infinite loops (`lo` never advances when `hi = mid` meets `mid = lo`), missing the first/last element, and index-out-of-range on empty inputs. `bisect_left` returns the *insertion point* — it may equal `len(a)`; check before indexing.

BFS vs DFS selection errors. Using DFS for "minimum number of steps" returns *a* path, not the shortest — a wrong-answer bug, not a crash, which makes it worse. Forgetting the visited set on a cyclic graph loops forever. Marking nodes visited when *popped* rather than when *pushed* lets BFS enqueue duplicates — correct answers, exponential queue.

Heap subtleties. `heapq` is min-only: negate for max behavior, and remember to negate again on the way out. Heap order is only guaranteed at the root — iterating a heap list does *not* yield sorted order; pop repeatedly or use `nsmallest`. Pushing unhashable/incomparable payloads without a tiebreaker key crashes on ties.

From-scratch ML implementation traps. In vectorized distance computation, the expansion $||a-b||^2 = ||a||^2 + ||b||^2 - 2a \cdot b$ can go slightly negative from floating-point error — clip at 0 before any `sqrt`. In k-means, forgetting the empty-cluster case crashes on `mean` of an empty slice; forgetting to seed the RNG makes results unreproducible, which interviewers notice. In KNN, taking `argsort(...)[:k]` instead of `argpartition` is correct but signals you don't know selection beats sorting. In reservoir sampling, `k/i` with integer division (a Python 2 habit, or `//`) silently samples nothing past item k.

What interviewers actually score. Restate the problem and confirm edge cases (empty input, k > n, ties, cycles) *before* coding. Name the brute force and its complexity, then improve it — jumping straight to the clever solution without the baseline reads as memorization. State time and space complexity unprompted at the end. And for ML candidates specifically: connecting the pattern to its ML instance ("this is a top-k heap — same thing beam search does") is the highest-signal sentence you can say in this round.

Interview questions and answers

Arrays, strings, hash maps

Q1. You need to find whether any two numbers in an unsorted array sum to a target. Give three approaches and their trade-offs.

Brute force: check all pairs, $O(n^2)$ time, O(1) space. Sort + two pointers: O(n log n) time, O(1) extra space (beyond the sort), but destroys original indices. Hash map (one pass, ask for `target - x`): O(n) time, O(n) space, preserves indices. The hash map is the default answer; mention the two-pointer variant when the array is already sorted or memory is tight. *Interviewers listen for: the space-time trade-off stated explicitly, not just the fastest answer.*

Q2. Why is building a string with += in a loop $O(n^2)$, and what is the fix?

Strings are immutable — each `+=` allocates a new string and copies both operands, so k appends of constant-size pieces copy $1+2+\ldots+k$ characters: $O(n^2)$ total. Fix: append pieces to a list and `''.join(parts)` once — one final allocation, O(n). (CPython has a narrow in-place optimization for unshared strings, but you must not rely on it and interviewers know it.)

Q3. Design an O(n) algorithm for "longest substring without repeating characters."

Sliding window with a hash map of char → last index. Expand `right`; if `s[right]` was seen at position ≥ `left`, jump `left` to one past that position; track the max window length. Each pointer only moves forward, so O(n) time, O(min(n, alphabet)) space. The invariant to state: the window `[left, right]` never contains a repeat.

Q4. What makes a good dict key, and why can't you use a NumPy array or a list?

Keys must be hashable: define `__hash__` and `__eq__`, and the hash must never change while the key is in the table. Lists and arrays are mutable, so their hash would go stale and lookups would miss — both are unhashable by design. Convert to `tuple(arr)` or `arr.tobytes()` (include shape/dtype if they vary). This bites in practice when memoizing feature computations keyed by input arrays.

Q5. How would you find the top-10 most frequent tokens in a 100 GB text corpus that doesn't fit in memory?

Frequency counting doesn't need the corpus in memory — it needs the *vocabulary* in memory: stream the file, update a Counter (usually fits; vocabularies are far smaller than corpora), then `most_common(10)` (a size-10 heap over vocab entries, O(V log 10)). If even the vocabulary overflows: hash-partition tokens across spill files so all occurrences of a token land in the same file, count each file separately, merge the per-file top-k candidates. *Interviewers listen for: distinguishing corpus size from vocab size, and the hash-partition trick.*

Stacks, queues, heaps

Q6. Check whether a string of brackets ()[]{} is balanced. What structure and why?

A stack — nesting is LIFO. Push openers; on a closer, the stack top must be the matching opener (pop it), else fail. Valid iff the stack is empty at the end. Edge cases to name: closer on empty stack, leftover openers. O(n) time, O(n) worst-case space. Generalizes to expression parsing and to validating nested JSON/HTML.

Q7. Why does "k largest elements of a stream" use a min-heap rather than a max-heap?

You need fast access to the *smallest of the current top-k* — the candidate to evict when something better arrives. A min-heap of size k keeps that at the root: compare each new item to the root, `heappushpop` if larger, O(log k) per item and O(k) memory. A max-heap of everything gives you the global max fast but eviction of the k-th best is what the problem needs. O(n log k) total vs O(n log n) for sorting.

Q8. How do you find the median of a data stream in O(log n) per element?

Two heaps: a max-heap of the lower half, a min-heap of the upper half, sizes kept within 1 of each other. Each insert goes into one heap and possibly rebalances one element across — O(log n); the median is a root (or the mean of both roots). This is the streaming form of quickselect's job, and the same two-heap idea underlies sliding-window medians for robust rolling statistics.

Q9. Why is deque the right structure for a BFS frontier and list the wrong one?

BFS pops from the front and pushes to the back. `deque.popleft()` is O(1); `list.pop(0)` shifts every remaining element, O(n), turning BFS's O(V+E) into O(V^2+E). A deque is a doubly-linked block structure with O(1) at both ends. Same reasoning applies to any producer-consumer buffer, e.g. batches queued between DataLoader workers and a training loop.

Q10. Where exactly does a heap appear in beam search?

At each decoding step you have (beam × vocab) candidate extensions and need the k best by cumulative log-probability — a top-k selection. A size-k min-heap over candidate scores does it in O(bV log k); in practice frameworks use `topk` (partition-based selection) on the score tensor, which is the same operation vectorized. The heap framing also explains beam search's memory: O(k) partial sequences, never the full search tree.

Trees, graphs

Q11. Why does inorder traversal of a BST yield sorted order, and what does that give you for validation?

The BST invariant puts everything smaller in the left subtree and everything larger in the right; inorder visits left subtree, node, right subtree, so by induction the output is sorted. Validation: run inorder and check the sequence is strictly increasing — O(n), and it catches the deep-violation case (a node that satisfies its parent but violates a distant ancestor) that per-node child checks miss. The bounds-passing version (Listing 4) does the same without materializing the sequence.

Q12. Predicting with a depth-d decision tree is O(d). Why can training-time tree depth blow up, and what limits it?

Prediction is one root-to-leaf descent: one threshold comparison per level. But an unconstrained tree keeps splitting until leaves are pure — on n distinct points it can grow to depth O(n) (a chain) when splits are unbalanced, e.g. sorted or pathological features. Limits: `max_depth`, `min_samples_leaf`, `min_impurity_decrease` — the same hyperparameters that control overfitting (Chapter 6), because a memorizing tree and a degenerate tree are the same object.

Q13. When do you choose BFS over DFS and vice versa? Give the memory trade-off too.

BFS when the question is "fewest edges/steps" — it explores in distance layers, so the first arrival is optimal in an unweighted graph. DFS for exhaustive exploration: components, cycle detection, backtracking, topological orderings. Memory: BFS holds a frontier that can be O(V) wide (bad on bushy graphs); DFS holds a path that can be O(V) deep (bad on long chains, plus Python's recursion limit — use an explicit stack). Weighted shortest paths need Dijkstra: BFS with a priority queue.

Q14. Your feature pipeline is a set of "A must run before B" constraints. How do you order it, and how do you detect an impossible configuration?

Model as a DAG and topologically sort. Kahn's algorithm: repeatedly emit a zero-in-degree node and decrement its neighbors' in-degrees, using a queue; O(V+E). If the algorithm stalls with nodes still un-emitted, those nodes form (or depend on) a cycle — the impossible configuration — and the leftover set localizes it for the error message. Same algorithm inside Airflow scheduling and autodiff's forward-pass ordering; backprop then runs the reverse order.

Q15. Count the connected components in a friendship graph with 10M nodes. What do you use and what breaks first?

Iterative DFS (or BFS) from every unvisited node, adjacency list, visited set: O(V+E) time, O(V) space. What breaks: recursive DFS hits Python's recursion limit on any long chain — must be an explicit stack; and at 10M nodes, per-object overhead of dict-of-lists may exhaust memory — move to array-based CSR adjacency or union-find (near-O(1) amortized per edge), which also handles the streaming-edges version. *Interviewers listen for: the recursion-limit fix and one scaling escape hatch.*

Sorting, searching, DP

Q16. Why is quicksort O(n log n) average but O(n²) worst case, and how is the worst case avoided?

Each partition costs O(n); the total is O(n × depth). Balanced partitions give depth O(log n); a pivot that is always the min/max (e.g. first element of sorted input) gives depth O(n), hence O(n²). Mitigations: random pivot (adversarial inputs become astronomically unlikely), median-of-three, or introsort (quicksort that falls back to heapsort past a depth budget — what C++ std::sort does). Python sidesteps it entirely: Timsort is a mergesort hybrid with guaranteed O(n log n) and O(n) on nearly-sorted data.

Q17. What is sort stability, and give a concrete ML-adjacent case where it matters.

A stable sort keeps equal keys in their original relative order. Case: event logs sorted by timestamp, then stable-sorted by user_id — each user's events remain time-ordered, which is required for building leakage-free sequential features or time-based splits; an unstable sort silently shuffles within-user order and corrupts them. Stability also composes multi-key sorts (secondary first, then primary). Python's `sorted` is stable; NumPy's default `np.sort` (quicksort) is not — pass `kind='stable'`.

Q18. You need the median of 1 billion floats. Full sort is too slow. Options?

Quickselect / `np.partition`: O(n) average, in memory. Out of memory: (1) two-heap streaming works for running medians but still holds all data; (2) distribution/bucket approach — histogram pass to find the bucket containing the median, second pass within it; (3) approximate quantile sketches (t-digest, GK) with bounded memory and error guarantees — the production answer for percentile monitoring at scale. *Interviewers listen for: naming that exact selection is O(n), then trading exactness for memory.*

Q19. What property must hold to "binary search on the answer," and how would you find the largest classification threshold with recall ≥ 0.9?

Monotonicity of the feasibility predicate in the searched parameter. Recall is monotone non-increasing as the threshold rises (fewer positives predicted), so "recall ≥ 0.9" flips from true to false exactly once — binary search the threshold in [0, 1] (or over the sorted unique scores, which is exact) checking recall at each mid: O(log n) recall evaluations instead of scanning all thresholds. Precision, by contrast, is not monotone in the threshold — say so before anyone asks.

Q20. State the edit-distance recurrence and explain what each branch means.

`d(i,j) = d(i-1,j-1)` if the current characters match (free), else `1 + min(d(i-1,j), d(i,j-1), d(i-1,j-1))` — delete from source, insert into source, substitute. Base cases `d(i,0)=i`, `d(0,j)=j`. O(mn) time; space compresses to two rows since each cell needs only the previous row and the current row's left neighbor. Used in spell-checkers, fuzzy joins in entity resolution; the same table shape as DTW, Viterbi, and CTC alignment.

Q21. Why is naive recursive Fibonacci exponential, and what are the two standard fixes? What's the general lesson?

fib(n) branches into fib(n−1) and fib(n−2), recomputing the same subproblems exponentially often — the call tree has $\sim\varphi^n$ nodes. Fixes: memoize the recursion (`@lru_cache` — top-down DP) or fill a table from the base cases (bottom-up, and keep only the last two values: O(1) space). Lesson: when a recursion has overlapping subproblems and optimal substructure, caching collapses exponential to polynomial — the definition of DP.

Q22. Your recursive solution is correct but crashes with RecursionError on large inputs. Enumerate your options.

In order: (1) convert to iteration — explicit stack for traversals, bottom-up table for DP (removes the limit and is usually faster); (2) restructure to reduce depth (e.g. recurse into the smaller side only, as in quickselect); (3) `sys.setrecursionlimit` — a last resort that trades the clean exception for a possible interpreter crash and doesn't fix the O(depth) frame memory. Python has no tail-call optimization, so tail-recursive style does not help — worth saying explicitly.

ML from scratch, sampling

Q23. Walk through vectorizing KNN's distance computation. Why not two nested loops?

Expand $||q-x||^2 = ||q||^2 + ||x||^2 - 2q \cdot x$: compute row-norms of both matrices, one matrix multiply `Q @ X.T`, and broadcast the norms into a (queries × train) distance matrix — all C-loop work, ~100× faster than Python loops (Chapter 2's boxing/dispatch argument). Then `argpartition` per row for the k smallest: O(n) selection, not an O(n log n) sort. Caveats worth volunteering: float error can make entries slightly negative (clip at 0 before sqrt), and the (q × n) matrix itself can exceed memory — chunk the queries.

Q24. Why is k-means guaranteed to converge, and to what?

Both steps monotonically decrease (or leave unchanged) the within-cluster sum of squares: reassigning a point to a nearer centroid lowers its contribution, and the mean is the unique minimizer of summed squared distance to a set of points. The objective is bounded below and there are finitely many assignments, so the loop must reach a fixed point — but only a *local* optimum depending on initialization. Hence k-means++ seeding and `n_init` restarts (Chapter 8). Complexity per iteration: O(nkd).

Q25. Prove reservoir sampling is uniform for k = 1.

Keep the first item; replace the current holder with item i with probability 1/i. Item i survives to the end iff it is accepted (1/i) and never displaced by any later j (each j fails to displace with probability $1 - 1/j = (j-1)/j$). P(item i final) = $(1/i) \times \prod_{j=i+1}^{n} (j-1)/j$ — the product telescopes to i/n — giving 1/n for every i. The k>1 proof is the same induction with k/i acceptance and uniform eviction. *Interviewers listen for: the telescoping product, stated cleanly.*

Q26. Implement weighted sampling given softmax probabilities. What are the complexity options?

Cumulative sums + binary search (Listing 12): O(n) setup, O(log n) per draw — right for many draws from a fixed distribution, and exactly what temperature sampling over a vocabulary does. Alternatives: linear scan of the CDF, O(n) per draw (fine for one draw); the alias method, O(n) setup and O(1) per draw (the answer to "can you beat log n?"); for one-shot vectorized use, `np.random.choice(n, p=probs)`. If weights update between draws (boosting, prioritized replay), a Fenwick/segment tree gives O(log n) updates and draws.

Q27. Why is "shuffle by sorting with a random comparator" wrong, and what is the correct algorithm?

A random comparator is inconsistent (violates transitivity), so the sort's behavior is undefined and the resulting permutation distribution is biased — famously demonstrated in a browser-ballot shuffle. Sorting by a random *key* per element is unbiased but O(n log n) and can collide. Correct: Fisher–Yates — swap position i with uniform j in [i, n), O(n), exactly uniform by induction. The `[0, n)` variant is the subtle trap: it produces n^n equally likely execution paths, which cannot map uniformly onto n! permutations since n! does not divide n^n.

Q28. Your 80/20 imbalanced dataset needs a train/test split. What goes wrong with a plain random split and what's the fix?

With a small test set, the minority class's test count is a random variable — it can land far from 20% (or at zero for rare classes), making test metrics noisy or undefined (recall with no positives). Fix: stratified split — group indices by label (hash-map grouping), shuffle within groups (Fisher–Yates), take the fraction from each: class proportions hold exactly (Listing 12). Same idea extends to stratified k-fold CV; for time series, stratification gives way to temporal splits — leakage beats balance (Chapter 4).

Q29. Estimate: KNN prediction cost for 1M training points, 512-dim embeddings, 1,000 queries, k=10. Then make it fast.

Brute force: 10^9 point-pairs × 512 mult-adds ≈ 5×10^{11} FLOPs — seconds as one big matrix multiply on good BLAS/GPU, but the 1000×1M distance matrix is ~4 GB in float32, so chunk queries. Per query it's O(nd), which kills low-latency serving. Faster: KD/ball trees fail at d=512 (curse of dimensionality — no pruning); the real answers are approximate nearest neighbor — inverted-file indexes, product quantization, HNSW graphs (Chapter 24) — trading exact recall for orders-of-magnitude speedups. *Interviewers listen for: memory of the distance matrix, and knowing when trees stop working.*

Q30. Design a data structure supporting insert, delete, and get-random in O(1), and say where the pattern shows up in ML.

Array + hash map: the array holds elements (random pick = uniform index, O(1)); the map holds value → array index. Delete: swap the target with the last element, update the moved element's map entry, pop — O(1) because only the last slot ever vacates. The pattern (dense array for sampling + index map for lookup) is how replay buffers and negative-sampling pools support uniform draws with O(1) membership updates.